

Pregunta 1: ¿Cómo transformaría esta aplicación hacia el paradigma orientado a objetos? ¿Cuántas clases agregaría? (dibuje diagrama de clases)

R Agregaría 1 clase llamada Matriz

Matriz

- datos: list
- + __init__(datos: list)
- + suma_pares() → int

Pregunta 2: Agregue al menos una funcionalidad que implique uso de herencia y sobreescritura de métodos (dibuje diagrama de clases)

R. Agrego la clase MatrizEstadistica que hereda de Matriz y sobreescribe suma_pares() para también imprimir el promedio de pares.

Matriz

- datos: list
- + __init__(datos: list)
- + suma_pares() → int

MatrizEstadistica

- + __init__(datos: list)
- + suma_pares() → int (*override*)

Pregunta 3: Codifique la aplicación en Python contemplando preguntas 1 y 2

```
class Matriz:
    def __init__(self, datos):
        self.datos = datos

    def suma_pares(self):
        suma = 0
        for fila in self.datos:
            for elemento in fila:
                if elemento % 2 == 0:
                    suma += elemento
        print(suma)
        return suma
```

```
class MatrizEstadistica(Matriz):
    def __init__(self, datos):
        super().__init__(datos)
```

```

def suma_pares(self): # sobreescritura
    suma = 0
    pares = []
    for fila in self.datos:
        for elemento in fila:
            if elemento % 2 == 0:
                suma += elemento
                pares.append(elemento)
    promedio = suma / len(pares) if pares else 0
    print(f"Suma pares: {suma}")
    print(f"Promedio pares: {promedio}")
    return suma

```

Uso

```

matriz = [
    [4, 1, 7],
    [2, 9, 3],
    [5, 8, 6]
]

```

```

m = MatrizEstadistica(matriz)
m.suma_pares()

```

Pregunta 4: ¿Cómo generaría un conversor de longitudes de millas a metros? ¿Qué debe tomar en cuenta? ¿Cómo lo aplicaría?

Factor de conversión: 1 milla = 1609.344 metros

Considerar: validación de entrada (no negativos, solo números), precisión decimal ya que se puede buscar en internet el valor exacto.

Aplicación: recibir el valor en millas, multiplicar por 1609.344 y retornar el resultado.

```

class Conversor:

```

```

    FACTOR = 1609.344

```

```

    def __init__(self, millas):
        if millas < 0:
            raise ValueError("El valor no puede ser negativo")
        self.millas = millas

```

```

    def convertir(self):
        return self.millas * self.FACTOR

```

```

class ConversorDetallado(Conversor):

```

```

    def __init__(self, millas):
        super().__init__(millas)

```

```
def convertir(self): # sobreescritura
    metros = super().convertir()
    print(f"{self.millas} millas = {metros:.2f} metros")
    return metros
```

```
c = ConversorDetallado(5)
c.convertir()
# resultado : 5 millas = 8046.72 metros
```